
Claude Registry

Guida Operativa

Infrastruttura di governance per capability condivise di Claude Code

Versione 2.0 — Maggio 2026

Luca La Rosa

`luca.la.rosa@accenture.com`

Indice

Prefazione	v
I Visione d'insieme	1
1 Il problema che risolve il registro	3
1.1 Il problema della conoscenza dispersa	3
1.2 La soluzione: un registro condiviso	3
1.3 Cosa non è il registro	4
2 Filosofia di progettazione	5
2.1 Due tipi di capability: agenti e skill	5
2.1.1 Agenti	5
2.1.2 Skill	5
2.2 Il principio di separazione catalogo/marketplace	6
2.3 La filosofia della conoscenza negativa	6
2.4 Human-in-the-loop by design	6
3 Struttura del repository	9
3.1 Mappa delle directory	9
3.2 Il file catalog.json	10
4 Installazione e primo avvio	11
4.1 Prerequisiti	11
4.2 Installazione globale	11
4.3 Installazione per progetto	11
4.4 Verifica dell'installazione	12
4.5 Aggiornamento	12
II Contribuire	13
5 Il flusso di contribuzione	15
5.1 Ruoli nel registro	15
5.2 Workflow per un nuovo agente	15
5.3 Ciclo di vita di una capability	16

5.4	Promozione da beta a stabile	16
5.5	Versionamento SemVer	17
6	Come scrivere un agente	19
6.1	Il formato del file	19
6.2	Il campo <code>description</code> — il più importante	20
6.3	Scelta del modello	20
6.4	Profili strumenti standard	20
6.5	Checklist pre-PR per un agente	21
7	Come scrivere una skill	23
7.1	Quando creare una skill	23
7.2	Il formato di una skill	23
7.3	Disclosure progressiva — limiti di lunghezza	24
7.4	Script deterministici opzionali	24
8	Governance e anti-pattern	25
8.1	Processo decisionale	25
8.1.1	Aggiungere una nuova capability	25
8.1.2	Modificare una capability esistente	25
8.1.3	Deprecare una capability	25
8.2	Catalogo vs. progetto	26
8.3	SLA del registro	26
III	Le capability del catalogo	27
9	Orchestrazione	29
9.1	<code>orchestrator</code> — Meta-orchestratore	29
9.2	<code>refactoring-supervisor</code> — Supervisore del <code>replatforming end-to-end</code>	30
9.2.1	Le 5 fasi del workflow	30
9.2.2	La Fase 4 — <code>Replatforming</code> progressivo	30
9.2.3	<code>Human-in-the-loop</code>	31
10	Deliberazione	33
10.1	<code>deliberative-decision-engine</code> — Motore deliberativo	33
10.1.1	Le priorità del motore	33
10.1.2	Il pipeline a 7 step	33
10.1.3	Trigger	34
10.1.4	Le persona del dibattito	34
10.1.5	Artefatto di output	34
11	Phase 0 — Indicizzazione	35
11.1	Il problema dell'indicizzazione	35
11.2	<code>indexing-supervisor</code> — Supervisore dell'indicizzazione	35
11.2.1	I 7 sub-agent	35
11.2.2	Stack awareness	36

12 Phase 1 — Analisi funzionale	37
12.1 functional-analysis-supervisor	37
12.1.1 I 3 wave di analisi	37
12.1.2 Resume mode exports-only	38
13 Phase 2 — Analisi tecnica	39
13.1 technical-analysis-supervisor	39
13.1.1 Gli 8 analisti (Wave 1)	39
13.1.2 Sintesi e challenger (Wave 2 e 3)	40
14 Phase 3 — Baseline testing	41
14.1 baseline-testing-supervisor	41
14.1.1 L'oracle	41
14.1.2 Failure policy	41
14.1.3 I sub-agent	41
15 Phase 4 — Refactoring TO-BE (agenti dedicati)	43
15.1 Agenti di decomposizione e contratto	43
15.1.1 decomposition-architect	43
15.1.2 api-contract-designer	43
15.2 Agenti di scaffolding	43
15.2.1 backend-scaffolder	43
15.2.2 data-mapper	43
15.2.3 logic-translator	44
15.2.4 frontend-scaffolder	44
15.3 Agenti di hardening e roadmap	44
15.3.1 hardening-architect	44
15.3.2 migration-roadmap-builder	44
15.3.3 phase4-challenger	44
16 Analisi funzionale e tecnica standalone	45
16.1 functional-analyst	45
16.2 technical-analyst	45
16.3 software-architect	45
17 Developer agents	47
17.1 Skill backend caricati da developer-java	47
18 Qualità	49
18.1 code-reviewer	49
18.2 debugger	49
18.3 test-writer	49
18.4 registry-auditor	49
19 Documentazione	51
19.1 documentation-writer	51
19.2 wiki-writer	51

19.3	document-creator	51
19.4	presentation-creator	51
19.5	Skill di documentazione	52
20	API design	53
20.1	api-designer	53
IV	Strumenti di contorno	55
21	Hooks	57
21.1	Cosa sono gli hook	57
21.1.1	Proprietà chiave	57
21.2	Hook inclusi nel catalogo	57
21.2.1	pre-tool-safety.sh	57
21.2.2	post-session-log.sh	58
21.3	Come attivare gli hook	58
21.4	Linee guida per lo sviluppo di hook	58
22	MCP (Model Context Protocol)	59
22.1	Cos'è MCP	59
22.2	Quando usare MCP — e quando no	59
22.2.1	Usa MCP quando	59
22.2.2	Non usare MCP quando	59
22.3	Formato del file .mcp.json	60
22.4	Server MCP comuni	60
23	Script operativi	61
23.1	setup-capabilities.sh	61
23.2	new-capability.sh	61
23.3	publish.sh	61
23.4	validate_catalog.py e validate_marketplace.py	62
23.4.1	Cosa valida Gate 1 (catalogo)	62
23.4.2	Cosa valida Gate 2 (marketplace)	62
24	Template e policy	63
24.1	Template	63
24.2	Policy	63
24.3	Evals	63
A	Riferimento rapido: comandi essenziali	65
B	Elenco completo delle capability	67
B.1	Agenti per area	67
C	Glossario	71

Prefazione

Questo documento è la guida operativa completa del **Claude Registry**: un'infrastruttura di governance per distribuire, versionare e condividere capability specializzate di Claude Code all'interno del team.

La guida è organizzata in quattro parti progressive. La prima parte risponde alla domanda “perché esiste il registro e come funziona”. La seconda spiega come contribuire. La terza documenta nel dettaglio tutte le capability disponibili, partendo dalle più architetturealmente complesse. La quarta copre gli strumenti di contorno: hooks, MCP, script e template.

Il tono è tecnico e diretto. Non è un tutorial introduttivo a Claude Code: si presuppone che il lettore sappia già usare Claude Code in un progetto. È invece la documentazione di riferimento per chi vuole capire il registro dall'interno — sia per usarlo al meglio, sia per estenderlo.

Parte I

Visione d'insieme

Capitolo 1

Il problema che risolve il registro

1.1 Il problema della conoscenza dispersa

Claude Code permette di definire sottoagenti personalizzati tramite file `.md` nella directory `.claude/agents/`. Ogni sottoagente ha un `system prompt`, un set di strumenti e un modello di linguaggio. Nella pratica del team, questo meccanismo produce un problema ricorrente: ogni progetto riscrive da zero le stesse capability — lo stesso agente Java con le stesse convenzioni, lo stesso agente per il code review, lo stesso skill sugli standard Spring Boot.

Il risultato è triplice: le versioni divergono silenziosamente tra un progetto e l'altro, le correzioni scoperte su un progetto non fluiscono agli altri, e ogni nuovo collaboratore deve capire da capo quale agente usare per quale task.

1.2 La soluzione: un registro condiviso

Il Claude Registry è la risposta a questo problema. Funziona esattamente come una libreria di codice condivisa, ma per il comportamento di Claude:

- **Fonte unica di verità:** le capability vivono in un repository Git versionato (`claude-catalog/`), non disperse nei singoli progetti.
- **Distribuzione controllata:** le capability approvate vengono pubblicate in `claude-marketplace` e installate nei progetti tramite uno script. I progetti consumano dal marketplace, non direttamente dal catalogo.
- **Governance esplicita:** ogni modifica passa per una Pull Request con checklist di revisione. Ogni capability ha esempi verificabili ed eval scenarios.
- **Conoscenza negativa preservata:** gli anti-pattern (approcci tentati che non hanno funzionato) sono documentati esplicitamente in `ANTI-PATTERNS.md`, in modo che il prossimo contributore non ripeta gli stessi errori.

1.3 Cosa non è il registro

Il registro non è:

- Un framework di prompting generico — contiene capability con opinioni forti su standard specifici (Spring Boot 3, Angular 17, OpenAPI 3.1).
- Un sostituto di Claude Code — è un layer di governance sopra le feature native di Claude Code.
- Un sistema di deployment autonomo — le capability vengono installate localmente nei progetti; non esiste un endpoint remoto condiviso.

Capitolo 2

Filosofia di progettazione

2.1 Due tipi di capability: agenti e skill

Il registro distingue nettamente due tipi di capability, con ruoli e vincoli molto diversi.

Tipo	Scopo	Modello	Strumenti	Posizione
Agente	Svolge un ruolo	sonnet	Set specifico al ruolo	agents/
Skill	Fornisce conoscenza	haiku	Solo Read	skills/

2.1.1 Agenti

Un agente è un sottoagente Claude Code che *fa* qualcosa: analizza, scrive codice, progetta architetture, esegue revisioni. Ha un system prompt con logica comportamentale, strumenti appropriati al ruolo, e una descrizione precisa che guida Claude nella delega automatica.

Gli agenti hanno opinioni forti: definiscono formati di output espliciti, seguono framework nominati (C4 per i diagrammi, RFC 7807 per gli errori HTTP, ADR per le decisioni architetturali), e specificano cosa delegano a loro volta ad altri agenti.

2.1.2 Skill

Una skill è un *fornitore di conoscenza*: standard di codifica, convenzioni di un framework, regole di branding, template riutilizzabili. È un nodo foglia — non delega ad altri agenti, non modifica file, non esegue comandi. Restituisce solo contenuto.

Il modello haiku è sufficiente perché non serve ragionamento profondo: serve recupero affidabile di informazioni già strutturate.

Buona pratica

Crea una skill ogni volta che la stessa conoscenza di dominio (standard di codifica, regole di design, costanti di brand) appare in due o più system prompt di agenti. Una skill è più economica da aggiornare di due copie separate.

2.2 Il principio di separazione catalogo/marketplace

Il repository ha due aree con responsabilità distinte:

Area	Scopo	Chi la modifica
claude-catalog/	Sviluppo — scrittura, revisione, versionamento	Sviluppatori, PR
claude-marketplace/	Distribuzione — solo capability approvate	Script di publish

Una capability non è “disponibile” finché non viene pubblicata nel marketplace. Modificare solo il catalogo non è sufficiente. Questo è il confine più importante da rispettare: **non modificare mai claude-marketplace/ a mano**.

2.3 La filosofia della conoscenza negativa

Ogni sistema di governance accumula saggezza sia in positivo (cosa funziona) che in negativo (cosa non funziona). Il registro esternalizza entrambe.

Il `CHANGELOG.md` registra cosa funziona e viene rilasciato. Il `ANTI-PATTERNS.md` registra cosa non ha funzionato e perché — con una clausola obbligatoria “*Do not retry unless*”. Questa clausola è la parte più importante: senza di essa, un futuro contributore potrebbe riproporre lo stesso approccio sotto un nome diverso, credendo di portare qualcosa di nuovo.

Attenzione

Aggiungere una entry in `ANTI-PATTERNS.md` è parte obbligatoria di ogni PR di deprecazione. Senza la entry, la deprecazione è incompleta.

2.4 Human-in-the-loop by design

I flussi più complessi del registro (refactoring end-to-end, analisi funzionale, analisi tecnica, baseline testing) sono progettati con un principio esplicito: Claude propone, l’utente decide.

Ogni fase mostra prima uno schematico di cosa sta per fare, poi aspetta conferma esplicita dell’utente prima di procedere. Al termine di ogni fase, produce un riepil-

logo con i tempi di esecuzione per fase e per step. Nessuna fase avanza silenziosamente senza consenso umano.

Questo non è un limite tecnico ma una scelta di design: i flussi di replatforming toccano codice di produzione e decisioni architettureali ad alto impatto. Il registro considera la supervisione umana un requisito non negoziabile per questi casi d'uso.

Capitolo 3

Struttura del repository

3.1 Mappa delle directory

claude-registry/	
claude-catalog/	
agents/	fonte di sviluppo
analysis/	file .md degli agenti
api/	analisi funzionale e tecnica standalone
architecture/	progettazione API
baseline-testing/	software architect
deliberation/	Phase 3 – pipeline di testing AS-IS (8 agenti)
developers/	motore di decisione deliberativa (7 agenti)
documentation/	agenti developer per linguaggio (9 agenti)
functional-analysis/	creazione documenti e presentazioni
indexing/	Phase 1 – analisi funzionale AS-IS (7 agenti)
orchestration/	Phase 0 – indicizzazione codebase (8 agenti)
quality/	orchestratore e refactoring-supervisor
refactoring-tobe/	code-reviewer, debugger, test-writer
technical-analysis/	Phase 4 – refactoring T0-BE (10 agenti)
tobe-testing/	Phase 2 – analisi tecnica AS-IS (11 agenti)
skills/	Phase 5 – testing equivalenza T0-BE (9 agenti)
analysis/	fornitori di conoscenza condivisa
api/	analisi funzionale e tecnica
backend/	standard REST
branding/	Java/Spring, standard tecnici backend
database/	Accenture branding (colori, font, template)
documentation/	PostgreSQL
frontend/	generatori di documentazione
orchestrators/	Angular, React, Vue, Qwik, Vanilla, CSS
python/	orchestratori backend e frontend
refactoring/	Python e Streamlit
testing/	principi di refactoring
utils/	standard di testing
docs/	utility (caveman, caveman-review, ecc.)
examples/	documentazione interna per contributori
evals/	invocazioni di esempio per capability
hooks/	scenari di valutazione
mcp/	script per hooks di Claude Code
policies/	esempi di configurazione MCP
	convenzioni di codifica (Python)

scripts/	
setup-capabilities.sh	installa capability in un progetto
new-capability.sh	scaffolda una nuova capability
templates/	template di output (ADR, report, contratto API)
how-to-write-a-capability.md	
CONTRIBUTING.md	
GOVERNANCE.md	
CHANGELOG.md	
ANTI-PATTERNS.md	
NAMING-CONVENTIONS.md	
claude-marketplace/	distribuzione
stable/<topic>/	capability production-ready
beta/<topic>/	capability nuove o sperimentali
skills/<topic>/	skill condivise
catalog.json	manifesto – fonte di verita' dei path su disco
guida-operativa.tex	questo documento (sorgente LaTeX)
guida-operativa.pdf	versione compilata
pitch-claude-registry.pptx	presentazione Accenture-branded

3.2 Il file catalog.json

Il file `claude-marketplace/catalog.json` è il manifesto del marketplace. Ogni capability pubblicata ha una entry con i campi:

- **name** — identificatore unico, deve corrispondere al frontmatter del file `.md`
- **version** — versione SemVer (MAJOR.MINOR.PATCH)
- **tier** — `stable`, `beta`, o `skill`
- **file** — percorso relativo del file su disco — fonte di verità per lo script di setup
- **status** — `active` o `deprecated`
- **dependencies** — lista di skill che l'agente usa (installate automaticamente dallo script)

Attenzione

Il campo `file` in `catalog.json` è l'unica fonte di verità per il percorso su disco. Lo script `setup-capabilities.sh` lo legge direttamente. Non modificare mai i path a mano al di fuori del publish script.

Capitolo 4

Installazione e primo avvio

4.1 Prerequisiti

- Claude Code CLI installato e configurato
- Git con accesso al repository del registro
- Python 3.8+ (per gli script di validazione)
- Bash 4+ (per gli script di setup)

4.2 Installazione globale

Per installare tutte le capability globalmente (disponibili in tutti i progetti senza configurazione aggiuntiva):

```
cd ~/dev/claude-registry
git pull origin main
./claude-catalog/scripts/setup-capabilities.sh --global
```

Lo script copia i file degli agenti in `~/claude/agents/` e le skill nelle rispettive directory. Le dipendenze (skill richieste da un agente) vengono installate automaticamente leggendo il campo `dependencies` in `catalog.json`.

4.3 Installazione per progetto

Per installare le capability solo in un progetto specifico:

```
./claude-catalog/scripts/setup-capabilities.sh /percorso/al/progetto
```

I file vengono copiati in `/percorso/al/progetto/claude/agents/`.

4.4 Verifica dell'installazione

Dopo l'installazione, verifica che gli agenti siano disponibili aprendo Claude Code nel progetto e digitando `/agents`. Dovresti vedere l'elenco degli agenti installati.

Per verificare che la versione installata corrisponda a quella pubblicata nel marketplace:

```
# Versione pubblicata nel marketplace
cat claude-marketplace/catalog.json | \
python3 -c "import json,sys; \
[print(c['name'], c['version']) \
for c in json.load(sys.stdin)['capabilities']]"
```

4.5 Aggiornamento

```
cd ~/dev/claude-registry
git pull origin main
./claude-catalog/scripts/setup-capabilities.sh --global
```

L'aggiornamento sovrascrive i file esistenti con le versioni più recenti. Non cancella capability rimosse — rimuovile manualmente se necessario.

Parte II

Contribuire

Capitolo 5

Il flusso di contribuzione

5.1 Ruoli nel registro

Ruolo	Responsabilità
Capability Author	Scriva nuovi agenti o skill; fornisce esempi ed eval
Capability Reviewer	Revisiona le PR usando <code>review-checklist.md</code>
Catalog Maintainer	Merge delle PR approvate, gestione tag Git, publish
Consumer	Usa le capability dal marketplace; apre issue per bug o gap

Un membro del team può ricoprire più ruoli. Non serve un team dedicato.

5.2 Workflow per un nuovo agente

```
# 1. Scaffolda la nuova capability
./claude-catalog/scripts/new-capability.sh --type agent nome-agente

# 2. Modifica il file generato
vim claude-catalog/agents/<topic>/nome-agente.md

# 3. Aggiungi esempi ed eval
vim claude-catalog/examples/nome-agente-example.md
vim claude-catalog/evals/nome-agente-eval.md

# 4. Aggiungi entry in CHANGELOG.md sotto [Unreleased]
vim claude-catalog/CHANGELOG.md

# 5. Valida localmente
python3 .github/scripts/validate_catalog.py

# 6. Pubblica nel marketplace (necessario per superare il gate CI)
./claude-marketplace/scripts/publish.sh nome-agente 0.1.0 beta

# 7. Valida il marketplace
python3 .github/scripts/validate_marketplace.py
```

```
# 8. Apri PR
git push origin add/nome-agente
```

Attenzione

La CI ha due gate sequenziali: prima valida il catalogo, poi — solo se il primo è verde — valida il marketplace. Se aggiungi una capability al catalogo senza pubblicarla nel marketplace, il gate 1 blocca la PR prima ancora che il gate 2 parta. Pubblica sempre prima di aprire la PR.

5.3 Ciclo di vita di una capability

```
draft -> review -> approved -> released -> published -> deprecated
```

Stato	Significato	Dove vive
draft	Lavoro in corso su feature branch	claude-catalog/ (branch)
review	PR aperta, revisione in corso	PR
approved	PR mergiata su main	claude-catalog/ (main)
released	Tag Git applicato	Git tag
published	Copiato nel marketplace	claude-marketplace/
deprecated	Non più raccomandato	Marketplace con notice

5.4 Promozione da beta a stable

Una capability beta è promovibile a stable quando soddisfa almeno uno dei seguenti criteri (alternativi, non cumulativi):

- **Criterio A — Tempo + adozione:** usata in almeno due progetti distinti senza problemi critici per almeno 30 giorni dall'ultimo rilascio beta.
- **Criterio B — Convergenza:** due o più specializzazioni di progetto indipendenti hanno introdotto la *stessa modifica* alla capability beta. La convergenza è un segnale più forte del tempo: se due team hanno indipendentemente aggiunto la stessa regola, quella regola appartiene alla capability globale, non ai progetti.

Per la promozione via Criterio B, la PR deve citare entrambe le specializzazioni con i percorsi dei file e il diff che dimostra la convergenza.

5.5 Versionamento SemVer

Tipo di cambiamento	Bump
Correzione di bug nel comportamento	PATCH (x.y.Z)
Nuovi comportamenti, nuovi formati di output	MINOR (x.Y.0)
Modifica a name o description nel frontmatter	MAJOR (X.0.0)

Attenzione

Cambiare il campo name o description nel frontmatter è un **breaking change**: modifica il modo in cui Claude decide di delegare a quell'agente. Richiede bump MAJOR e una nota di migrazione in `CHANGELOG.md`.

Capitolo 6

Come scrivere un agente

6.1 Il formato del file

Un agente è un file `.md` con frontmatter YAML seguito dal system prompt:

```
---
name: nome-agente
description: >
  Use when <trigger condition>. <cosa fa>. <cosa NON fa>.
  Typical triggers include "<trigger 1>", "<trigger 2>".
  See "When to invoke" in the agent body for worked scenarios.
tools: Read, Grep, Glob, Write, Edit, Bash
model: sonnet
color: blue
---

## Role

Un paragrafo che definisce chi e' questo agente e per cosa
e' autorevole.

## When to invoke

- **Scenario 1.** Descrizione del trigger e del comportamento atteso.
- **Scenario 2.** Altro trigger tipico.

Do NOT use this agent for: <casi esclusi>.

## What you always do

Lista di comportamenti obbligatori in ogni interazione.

## What you never do

Divieti espliciti con motivazione quando non ovvia.

## Output format

Definizione esatta del formato prodotto.
```

```
## Quality criteria
```

```
Come l'agente valuta il proprio output prima di rispondere.
```

6.2 Il campo `description` — il più importante

La `description` controlla *quando* Claude delega a questo agente. Deve:

- Iniziare con “Use when” o “Use for” (o la variante aggressiva “ALWAYS use when” per agenti sottoutilizzati)
- Specificare la condizione di trigger, non solo il dominio
- Includere 2–3 frasi verbatim di trigger tra virgolette doppie
- Contenere una clausola “Do not use for” esplicita per disambiguare rispetto ad agenti simili
- Terminare con il pointer: See “When to invoke” in the agent body for worked scenarios.

6.3 Scelta del modello

Modello	Quando usarlo
sonnet	Default — analisi, scrittura codice, documentazione
opus	Solo quando la profondità di ragionamento è critica
haiku	Skill e task ad alto volume e bassa complessità

Gli agenti con `model: opus` devono avere un campo `model_justification` nel `front-matter` con almeno 40 caratteri che spiegano la necessità di `reasoning depth` superiore. Senza questo campo, il validatore emette un warning.

6.4 Profili strumenti standard

Profilo	Strumenti	Uso tipico
Analista read-only	Read, Grep, Glob	Analisi senza output scritto
Analista con report	Read, Grep, Glob, Write	Analisi con deliverable
Developer completo	Read, Edit, Write, Bash, Grep, Glob	Scrittura ed esecuzione
Ricercatore	Read, Grep, Glob, WebFetch	Design con consultazione web
Orchestratore	Read, Glob, Agent	Solo delega ad altri agenti

6.5 Checklist pre-PR per un agente

- ☐ Il campo `name` corrisponde al nome del file (senza `.md`)
- ☐ `description` inizia con “Use when” o “Use for”
- ☐ Il `system prompt` definisce un ruolo chiaro, non un helper generico
- ☐ I formati di output sono definiti esplicitamente
- ☐ La lista degli strumenti è minimale (non * senza giustificazione)
- ☐ Nessuna credenziale o segreto nel file
- ☐ Almeno un esempio in `examples/`
- ☐ Almeno due scenari `eval` in `evals/`
- ☐ Il file è stato testato localmente almeno una volta
- ☐ `CHANGELOG.md` ha una entry sotto `[Unreleased]`
- ☐ `catalog.json` è aggiornato (dopo publish)

Capitolo 7

Come scrivere una skill

7.1 Quando creare una skill

Crea una skill quando:

- La stessa conoscenza di dominio appare in due o più system prompt di agenti
- La conoscenza è dichiarativa: descrive *come* fare le cose, non *cosa* fare
- La conoscenza cambia indipendentemente dal comportamento di ogni singolo agente

Non creare una skill per:

- Logica specifica di un singolo agente
- Comportamenti che richiedono strumenti oltre Read
- Contenuto già abbastanza corto da essere inlineato senza duplicazione

7.2 Il formato di una skill

```
---
name: nome-skill
description: >
  Use to retrieve [dominio] standards: [lista topic].
  Called by [agente-1] and [agente-2] before [task].
  Returns structured reference material, not code.
tools: Read
model: haiku
color: cyan
---

## Role

Sei un knowledge provider per gli standard di [dominio].
Quando invocato, restituisci gli standard completi
rilevanti per il task.
```

Standards

[Il contenuto – organizzato per topic, con regole ed esempi]

7.3 Disclosure progressiva — limiti di lunghezza

Livello	Contenuto	Quando caricato
Frontmatter	name, description, tools, model	Sempre
Body skill	Sezioni ## Role e ## Standards	Quando l'agente invoca la skill
references/	Dettagli lunghi, template completi, esempi esaustivi	Solo quando il body li linka e

Limite	Parole	Azione
Soft cap	3 000	Review se materiale denso può spostarsi in references/
Hard cap	5 000	Obbligatorio: regole nel body, template/esempi in references/

7.4 Script deterministici opzionali

Una skill può includere una directory scripts/ con script per logica deterministica. La regola è: se la regola suona come “dato X, la risposta è *sempre* Y”, appartiene a uno script. Se suona come “dato X, considera Y bilanciando Z”, rimane in prosa.

```

claude-catalog/skills/<topic>/<skill-name>/
  <skill-name>.md          body della skill
  references/              contenuto a disclosure progressiva
  scripts/
    README.md             indice degli script (obbligatorio)
    validate-something.py uno script, un compito, deterministico

```


Capitolo 8

Governance e anti-pattern

8.1 Processo decisionale

8.1.1 Aggiungere una nuova capability

L'autore apre una PR con: il file dell'agente/skill, almeno un esempio, almeno due scenari eval. Serve un'approvazione di un reviewer per il merge. Prima dell'apertura PR, il contributore deve cercare in `ANTI-PATTERNS.md` le parole chiave del ruolo proposto per verificare che non esista un precedente di fallimento.

8.1.2 Modificare una capability esistente

PR con diff, entry nel changelog, verifica degli eval. Una approvazione. Se il cambiamento modifica sensibilmente il comportamento del modello, bump MINOR.

8.1.3 Deprecare una capability

La PR di deprecazione deve fare tre cose *insieme*:

1. Aggiungere notice di deprecazione nel campo `description` del frontmatter e in `catalog.json`.
2. Aggiungere entry in `ANTI-PATTERNS.md` con: cosa è stato tentato, perché ha fallito, cosa lo ha sostituito, condizioni per un retry valido.
3. Tenere il file `.md` in posizione per 90 giorni prima della rimozione.

Il passo 2 è obbligatorio. Senza di esso, la rationale sparisce con il file e il prossimo contributore non ha modo di imparare dal precedente.

8.2 Catalogo vs. progetto

	Catalogo globale	Override di progetto
Dove	claude-catalog/ → market-place	<progetto>/claude/agents/
Contenuto	Orizzontale, riutilizzabile, technology-aware ma non domain-aware	Copia specializzata del catalog agent con regole di dominio specifiche
Business rules	Nessuna regola di business specifica	Aggiunge contesto di dominio (e.g., “questo servizio usa CQRS con Axon Framework”)
Promozione	—	Se l’override cresce molto, promuovere al catalogo

8.3 SLA del registro

- Revisione PR: entro 2 giorni lavorativi
- Triage di bug report su capability pubblicate: entro 5 giorni lavorativi
- Rimozione capability deprecate: 90 giorni dal notice di deprecazione

Parte III

Le capability del catalogo

Capitolo 9

Orchestrazione

9.1 orchestrator — Meta-orchestratore

L'orchestrator è il punto di ingresso per task che attraversano più domini simultaneamente o hanno scope ambiguo. Non scrive codice, non produce contenuto direttamente. Il suo valore sta in tre operazioni:

1. Capire la forma completa di una richiesta complessa prima che il lavoro inizi
2. Delegare gli agenti specialisti nel giusto ordine, in parallelo dove possibile
3. Sintetizzare i loro output in un'unica risposta coerente

Modello: opus — la profondità di ragionamento serve per riconciliare trade-off cross-domain e recuperare da fallimenti parziali di sub-agent.

Quando usarlo:

- Task multi-dominio: backend + frontend + database + documentazione simultaneamente
- Scope ambiguo: “migliora questa app”, “ottimizza le performance in tutto il progetto”
- Refactor cross-stack dove un cambio (rinomina DTO, forma endpoint, colonna DB) si propaga su più superfici
- Deliverable eterogeneo che nessun singolo agente può produrre da solo

Non usare per: task su un singolo dominio (usa lo specialista direttamente) o flussi di migrazione/refactoring end-to-end (usa refactoring-supervisor).

9.2 refactoring-supervisor — Supervisore del replatforming end-to-end

Il refactoring-supervisor è l'agente più complesso del registro: coordina un flusso di lavoro end-to-end di *application replatforming* su 5 fasi, ciascuna delegata a un supervisore di fase dedicato.

Nota

Il nome tecnico refactoring-supervisor è mantenuto per compatibilità backward, ma il capability name corretto è **Application Replatforming Workflow Supervisor**. La denominazione “refactoring” è obsoleta dalla v2.0.0.

9.2.1 Le 5 fasi del workflow

Fase	Supervisore	Output
Phase 0 — Indicizzazione	indexing-supervisor	.indexing-kb/
Phase 1 — Analisi funzionale	functional-analysis-supervisor	docs/analysis/01-functional
Phase 2 — Analisi tecnica	technical-analysis-supervisor	docs/analysis/02-technical
Phase 3 — Baseline testing	baseline-testing-supervisor	tests/baseline/
Phase 4 — Replatforming	(guidato direttamente)	Applicazione TO-BE funzionante

Le Fasi 0–3 sono **strictly AS-IS**: non referenziano mai tecnologie target né architetture TO-BE. La Fase 4 introduce il target tech (Spring Boot 3 + Angular per il default stack) e applica la regola di *inverse drift* (nessuna referenza AS-IS-only nel design TO-BE senza ADR).

9.2.2 La Fase 4 — Replatforming progressivo

La Fase 4 è sostanzialmente diversa dalle precedenti: invece di delegare a un singolo supervisore, il refactoring-supervisor la gestisce direttamente attraverso un loop a 7 step:

1. **Step 0 — Bootstrap** (HARD GATE): il progetto deve compilare e l'applicazione deve avviarsi
2. **Step 1 — Skeleton minimale**: struttura base funzionante
3. **Step 2 — Feature loop incrementale**: una feature alla volta — implementa → test → build → run → valida
4. **Step 3 — Mandatory validation loop**: qualsiasi fallimento blocca l'avanzamento fino alla risoluzione della causa radice
5. **Step 4 — Costruzione progressiva del sistema**: slices verticali, sistema sempre buildabile/runnable/testabile

6. **Step 5 — Hardening:** sicurezza, configurazione, error handling, logging reintrodotti e ri-validati
7. **Step 6 — Final validation:** suite di test completa + validazione dei business flow

Buona pratica

L'applicazione è **sempre in uno stato funzionante** durante la Fase 4. Nessun big-bang rewrite, nessun fallimento in fase avanzata, nessuno stato intermedio non compilabile.

9.2.3 Human-in-the-loop

Prima di ogni fase (e di ogni step della Fase 4), il supervisore mostra uno schematico di cosa sta per fare e attende conferma esplicita. Al termine di ogni fase produce un riepilogo con i tempi di esecuzione. Alla rilevazione di output esistenti chiede esplicitamente per-fase: skip / re-run / regenerate-exports / revise — mai auto-skip silenzioso.

Capitolo 10

Deliberazione

10.1 deliberative-decision-engine — Motore deliberativo

Il `deliberative-decision-engine` è l'agente per decisioni complesse, ad alto rischio, irreversibili o rilevanti per il replatforming. Non produce direttamente la risposta di dominio: orchestra un dibattito strutturato multi-agente e sintetizza il suo esito in un artefatto di decisione difendibile.

10.1.1 Le priorità del motore

1. **Qualità della decisione** — non degrada mai silenziosamente a ragionamento single-agent
2. **Robustezza** — porta in superficie le obiezioni non risolte; non le nasconde
3. **Auditabilità** — ogni fase produce un artefatto ispezionabile
4. **Explicabilità** — la risposta finale dichiara: cosa, perché, alternative considerate, obiezioni, dissensi, rischi residui, piano di validazione, piano di rollback
5. **Sicurezza** — per decisioni compliance/security/privacy/irreversibili escalate a judge o arbitrato umano

10.1.2 Il pipeline a 7 step

1. Trigger detection + classificazione del task
2. Decision framing
3. Draft indipendenti per 3 o 5 persona (anti-anchoring: nessuna persona vede le bozze altrui)
4. Summary strutturato delle evidenze (neutro — il judge non fa raccomandazioni)

5. 1–2 round di challenge
6. Rebuttal
7. Selezione della strategia finale (majority / confidence-weighted / consensus / judge / arbitrazione umana) + commit protocol + artefatto di audit

10.1.3 Trigger

Il motore si attiva per:

- Richiesta esplicita dell'utente in italiano: “decidi con dibattito”, “usa multi-agente”, “fai criticare la decisione”, “valuta pro e contro”
- Richiesta esplicita in inglese: “debate mode”, “multi-agent debate”, “red team this decision”
- Invocazione programmatica da refactoring-supervisor con decisionMode: deliberative

10.1.4 Le persona del dibattito

Agente	Ruolo
debate-proposer	Primary Architect — produce la raccomandazione principale
debate-critic	Skeptical Critic — audit delle assunzioni, failure modes
debate-replatforming-specialist	Migration Specialist — sequencing, cutover, rollback
debate-risk-reviewer	Risk/Compliance Reviewer — GDPR, HIPAA, blast radius
debate-operations-reviewer	Operations Reviewer — SLOs, RTO/RPO, costi operativi
debate-judge	Neutral judge — summary in Step 3 + arbitrazione in Step 6

Tutti i persona usano opus — la qualità della decisione è prioritaria rispetto al costo.

10.1.5 Artefatto di output

Il risultato del dibattito viene salvato in `.deliberation-kb/<trace-id>/` con la struttura completa del pipeline. Il report finale per l'utente include esplicitamente: decisione, rationale, alternative considerate, obiezioni, opinioni dissenzianti, rischi residui, piano di validazione, piano di rollback, e se è richiesta approvazione umana.

Capitolo 11

Phase 0 — Indicizzazione

11.1 Il problema dell'indicizzazione

Prima di analizzare o replatformare una codebase legacy, Claude ha bisogno di una *knowledge base strutturata* del codice: struttura delle directory, dipendenze, moduli, logica di business, flussi di dati. Leggere direttamente i file a runtime ad ogni query è costoso e inconsistente. L'`indexing-supervisor` risolve questo producendo una KB in markdown persistente su disco.

11.2 `indexing-supervisor` — Supervisore dell'indicizzazione

L'`indexing-supervisor` è l'entrypoint unico della Phase 0. Legge la codebase Python (con supporto opzionale Streamlit), dispatcha 7 sub-agent in 4 fasi, e produce la KB in `.indexing-kb/`.

11.2.1 I 7 sub-agent

Sub-agent	Cosa produce
<code>codebase-mapper</code>	Inventario strutturale: directory tree, LOC, package map, entrypoint
<code>dependency-analyzer</code>	Dipendenze esterne + grafo delle importazioni interne + circolarità
<code>streamlit-analyzer</code>	Pagine, <code>session_state</code> , widget, caching, pattern di navigazione Streamlit
<code>module-documenter</code>	Documentazione API per package (fan-out parallelo)
<code>data-flow-analyst</code>	DB, API esterne, file I/O, env vars, configurazione

Sub-agent	Cosa produce
business-logic-analyst	Regole di business, logica di validazione, glossario, state machines
synthesizer	Sintesi finale: overview di sistema, bounded contexts, hotspot di complessità

Nota

I sub-agent non sono invocati direttamente dall'utente. Il supervisore li dispatcha e li coordina. L'utente interagisce solo con l'`indexing-supervisor`.

11.2.2 Stack awareness

L'`indexing-supervisor` rileva il linguaggio primario e i framework dalla codebase e inietta istruzioni condizionali nei prompt dei sub-agent. Per Streamlit in particolare, lo `streamlit-analyzer` analizza il modello reattivo `script-as-page` che non ha un equivalente diretto in altri framework.

Capitolo 12

Phase 1 — Analisi funzionale

12.1 functional-analysis-supervisor

L'entrypoint unico della Phase 1. Legge `.indexing-kb/` (prodotto dalla Phase 0) e produce una comprensione funzionale completa dell'applicazione AS-IS in `docs/analysis/01-functional-analysis-supervisor/` più un export PDF e PPTX brandizzati Accenture.

La Phase 1 è **strictly AS-IS**: l'agente rifiuta qualsiasi riferimento a tecnologie target o architetture TO-BE. Se l'utente chiede analisi orientata al target, il supervisore risponde educatamente che quella è competenza delle fasi successive.

12.1.1 I 3 wave di analisi

Wave	Sub-agent	Output
W1	actor-feature-mapper	Attori, ruoli, persona, feature map, matrice Attore×Feature
W1	ui-surface-analyst	Schermate, grafo di navigazione, component tree
W1	io-catalog-analyst	Input/output, matrice di trasformazione
W2	user-flow-analyst	Use case, user flow, diagrammi Mermaid
W2	implicit-logic-analyst	Logica nascosta: validazione implicita, state machine, callback chain
W3	functional-analysis-challenger	Review avversariale (opt-in per stack non-Streamlit)

Dopo le 3 wave, il supervisore lancia in parallelo `document-creator` e `presentation-creator` per gli export Accenture-branded.

12.1.2 Resume mode exports-only

Se l'analisi funzionale è già completa su disco ma mancano i file PDF/PPTX, il supervisore rileva la situazione e offre di rigenerare solo gli export senza ri-eseguire l'analisi. Questo previene re-run costosi quando la KB non è cambiata.

Capitolo 13

Phase 2 — Analisi tecnica

13.1 technical-analysis-supervisor

Supervisore della Phase 2. Legge `.indexing-kb/` e (opzionalmente) `docs/analysis/01-functional` e produce una comprensione tecnica completa dell'applicazione AS-IS in `docs/analysis/02-technical` più export PDF e PPTX.

13.1.1 Gli 8 analisti (Wave 1)

Sub-agent	Analisi
code-quality-analyst	Struttura codebase, duplicazioni, hotspot di complessità, smell di monolite
state-runtime-analyst	Session state, global state, side effect, diagramma state-flow
dependency-security-analyst	Inventario dipendenze, CVE, deprecation watch, postura licenze, SBOM-lite
data-access-analyst	Flusso dati, pattern DB/file/cache/serializzazione
integration-analyst	Integrazioni esterne, auth/timeout/retry, mappa di integrazione
performance-analyst	N+1, hot loop, blocking I/O, gap di caching (analisi statica)
resilience-analyst	Error handling, logging, silent failure, fallback chain
security-analyst	OWASP Top 10, validazione input, segreti nel codice, STRIDE threat model

13.1.2 Sintesi e challenger (Wave 2 e 3)

Il risk-synthesizer (W2) consolida gli output degli 8 analisti in un registro di rischio unificato MD/JSON/CSV con matrice di severity e priorità di remediation.

Il technical-analysis-challenger (W3, sempre attivo) effettua una review avversariale per gap, contraddizioni e violazioni AS-IS.

Un singolo difetto spesso attraversa più domini (sicurezza + osservabilità + runtime): il supervisore usa opus per sintetizzare pattern cross-cutting visibili solo ragionando su tutti gli 8 output di W1.

Capitolo 14

Phase 3 — Baseline testing

14.1 baseline-testing-supervisor

Supervisore della Phase 3. Produce la baseline di regressione AS-IS: una suite pytest completa in `tests/baseline/`, snapshot oracle, benchmark baseline, e opzionalmente una Postman collection se l'app espone servizi HTTP.

14.1.1 L'oracle

L'oracle è il cuore della Phase 3. È l'insieme degli snapshot catturati dell'applicazione AS-IS (output per ogni use case, metriche di performance, contratti HTTP) che la Phase 5 userà come riferimento per verificare l'equivalenza funzionale del TO-BE.

Attenzione

Il `baseline-testing-supervisor` **non modifica mai il codice sorgente AS-IS**. Se un test fallisce per un bug latente nella codebase, lo gestisce secondo la failure policy — non corregge la fonte.

14.1.2 Failure policy

- Fallimenti **critical/high**: escalation all'utente
- Fallimenti **medium/low**: `@pytest.mark.xfail` con nota AS-IS bug

14.1.3 I sub-agent

Sub-agent	Ruolo
<code>fixture-builder</code>	<code>conftest.py</code> con <code>seed/time/network</code> determinism + fixture minimal/realistic/edge
<code>usecase-test-writer</code>	Modulo pytest per use case (happy + alternative + edge), Streamlit AppTest dove applicabile

Sub-agent	Ruolo
integration-test-writer benchmark-writer	Test DB / file system / API outbound (mocked) pytest-benchmark + profiling memoria — oracle performance AS-IS
service-collection-builder	Postman 2.1 collection (condizionale — solo se servizi esposti)
baseline-runner	Esegue la suite, cattura snapshot/benchmark JSON/coverage
baseline-challenger	Review avversariale (coverage, integrità oracle, determinismo)

Capitolo 15

Phase 4 — Refactoring TO-BE (agenti dedicati)

La Phase 4 è gestita direttamente dal refactoring-supervisor tramite il loop a 7 step descritto nel Cap. 10. Gli agenti documentati in questo capitolo supportano quei step e possono essere usati anche standalone.

15.1 Agenti di decomposizione e contratto

15.1.1 `decomposition-architect`

Decomposizione bounded-context (DDD), aggregati, mappa AS-IS↔TO-BE dei moduli, ADR-001 (stile architetturale) + ADR-002 (stack target).

15.1.2 `api-contract-designer`

Contratto OpenAPI 3.1 (singola fonte di verità), Postman collection TO-BE, ADR-003 (auth flow).

15.2 Agenti di scaffolding

15.2.1 `backend-scaffolder`

Spring Boot 3 Maven scaffold: controller da OpenAPI, DTO, service con TODO, error handler RFC 7807, security baseline.

15.2.2 `data-mapper`

Entità JPA, value object, enum, changelog Liquibase YAML, repository Spring Data JPA (DDD-compliant).

15.2.3 **logic-translator**

Fan-out per use case: traduce un singolo UC da Python AS-IS a Java/Spring. Non tocca mai il codice AS-IS.

15.2.4 **frontend-scaffolder**

Angular 17+ workspace: standalone component, lazy module per bounded context, client TypeScript generato da OpenAPI, traduzioni da Streamlit.

15.3 **Agenti di hardening e roadmap**

15.3.1 **hardening-architect**

Osservabilità (JSON logging + correlation-id, Micrometer + Prometheus, OpenTelemetry) e sicurezza (Spring Security 6, OWASP headers, CSP). ADR-004 + ADR-005.

15.3.2 **migration-roadmap-builder**

Roadmap strangler-fig con milestone per bounded context, piani di rollback, criteri di go-live, carry-over dei bug AS-IS.

15.3.3 **phase4-challenger**

Matrice di traceability AS-IS↔TO-BE + 8 check avversariali (coverage, OpenAPI↔code drift, completezza ADR, regression di sicurezza, equivalenza, leak AS-IS-only).

Capitolo 16

Analisi funzionale e tecnica standalone

16.1 functional-analyst

L'agente standalone per l'analisi funzionale su un progetto generico (non necessariamente parte di un flusso di replatforming). Estrae requisiti funzionali da specifiche, user story, o codice esistente; documenta use case e processi di business; produce criteri di accettazione; mappa attori e boundary di sistema. Genera anche CRUD matrix e traceability requirements→codice.

16.2 technical-analyst

L'agente standalone per l'audit tecnico: debito tecnico, postura di sicurezza, dipendenze vulnerabili, gap di osservabilità, metriche di qualità del codice, valutazione CI/CD pipeline. Produce findings strutturati con severity rating e priorità di remediation. **Non** fa raccomandazioni architetturali — delega a software-architect per questo.

16.3 software-architect

Analisi e progettazione di architetture di sistema, valutazione di scelte tecnologiche, review di pattern di integrazione, scrittura di Architecture Decision Record (ADR), assessment di requisiti non funzionali (performance, sicurezza, scalabilità, reliability, costo, manutenibilità). Usa il modello C4 per i diagrammi. Non scrive codice di implementazione.

Capitolo 17

Developer agents

Il registro include agenti developer specializzati per linguaggio. Ogni agente carica le skill di standard rilevanti per il linguaggio e il framework, e produce codice production-ready con testing, logging strutturato e error handling appropriato.

Agente	Stack	Note
developer-java	Java 17+ / Spring Boot 3	Standard enterprise complete,
developer-python	Python 3.11+ / FastAPI	PEP 8, type hints, Pydantic v2
developer-frontend	Angular / React / Vue / Qwik / Vanilla	Auto-rileva il framework del
developer-go	Go (net/http, chi, gin, cobra)	Standard library first, log/slo
developer-kotlin	Kotlin / Spring Boot 3 o Ktor	Coroutine, sealed hierarchy, v
developer-rust	Rust stable (axum, actix-web, tokio)	thiserror/anyhow, structured
developer-csharp	C# / .NET 8+ / ASP.NET Core	Nullable ref types, records, IC
developer-ruby	Rails 7+ / Sidekiq	Service/form/query objects, l
developer-php	PHP 8.2+ / Laravel 10+ o Symfony 6+	strict_types, readonly, enums,

17.1 Skill backend caricati da developer-java

- java-spring-standards — package structure, layering, testing, error handling, logging, security, Micrometer
- spring-expert — IoC/DI, auto-configuration, profiles, ConfigurationProperties, WebClient, Spring Security 6
- spring-architecture — Controller/Service/Repository/Entity, DTO, Bean Validation, mapper pattern
- spring-data-jpa — entity design, relazioni, fetch strategy, N+1, transazioni, JPQL, cache L2
- java-expert — records, sealed class, Optional, Stream API, Lombok, concurrency, custom exception

- postgresql-expert — data modelling, index, performance tuning, Flyway migrations, integrità referenziale

Capitolo 18

Qualità

18.1 code-reviewer

Code review strutturata su PR o set di file modificati. Analizza correttezza, vulnerabilità di sicurezza, test coverage, aderenza alle convenzioni di progetto, problemi di performance, manutenibilità. Produce una review con commenti a livello di riga e una raccomandazione overall: Approve, Request Changes, o Comment.

18.2 debugger

Diagnosi di bug, errori, o comportamento inatteso. Legge messaggi di errore, stack trace, log, e file sorgente rilevanti per identificare la causa radice e proporre una fix minimale e mirata. Non refactora oltre il necessario per correggere il bug. Spiega la causa radice chiaramente prima di proporre la fix.

18.3 test-writer

Scrittura di test per codice esistente: unit test, integration test, e2e test. Legge il codice di produzione, identifica scenari di test (happy path, edge case, error case), e produce test completi e runnable. Supporta JUnit 5 + Mockito (Java), pytest (Python), e Jest (JavaScript/TypeScript). Rileva e colma gap nelle suite di test esistenti.

18.4 registry-auditor

Audit del registro stesso contro le quattro rubric ufficiali di Anthropic (agent-development, skill-development, skill-creator, claude-md-improver). Produce: grade per area, pattern registry-wide, top-10 file da riscrivere, template di riferimento, outlier di word-count, quick win scriptabili.

Nota

registry-auditor è read-only: analizza ma non modifica nulla. È lo strumento principale per il loop audit→remediation: esegui l'auditor, leggi la top-10, correggi i file, ri-esegui il validatore, ri-esegui l'auditor per confermare che la metrica si sia mossa.

Capitolo 19

Documentazione

19.1 documentation-writer

Scrittura e miglioramento di documentazione tecnica: README, guide API, architecture overview, runbook, guide di onboarding, documentazione inline. Legge la codebase e la documentazione esistente per produrre documentazione accurata e adeguata all'audience.

Formati supportati: negozia il formato di output con l'utente prima di generare — supporta Markdown, L^AT_EX, HTML, PDF (via pandoc + pdflatex), DOCX. Rileva il toolchain locale e offre solo i formati effettivamente producibili.

19.2 wiki-writer

Scrittura di wiki GitHub organizzate secondo il framework Diataxis (Tutorials, How-to, Reference, Explanation). Produce file wiki/*.md per revisione via PR. Non fa auto-push alla wiki.

19.3 document-creator

Creazione di documenti tecnici Accenture-branded (PDF via HTML→Chrome headless, o DOCX) da documenti di progetto, file di stima, o materiali sorgente. Carica la skill accenture-branding per palette, tipografia e template. Produce: executive summary, problem statement, solution design, architettura, inventario componenti, dipendenze, timeline, rischi.

19.4 presentation-creator

Creazione di presentazioni .pptx Accenture-branded da documenti di progetto. Carica accenture-branding. Gestisce sia deck business (executive summary, proble-

ma/soluzione, timeline) sia deck tecnici (architettura, pattern, dipendenze, topologia cloud). Read-only sui sorgenti.

19.5 Skill di documentazione

Skill	Produce
backend-documentation	backend-doc.tex per Spring Boot (architettura, API, data model)
frontend-documentation	frontend-doc.tex per Angular (moduli, NgRx, routing, design)
documentation-orchestrator	Coordina i due precedenti + consistenza cross-layer
doc-expert	Docstring, flow description, glossario di dominio — salva in d
functional-document-generator	Converte docs/functional/ in .tex deliverable per stakeholder
uml-diagram-generator	Genera diagrammi UML (component, sequence, class, ecc.)

Capitolo 20

API design

20.1 **api-designer**

Design e review di contratti API REST: modellazione risorse, selezione metodi HTTP e status code, struttura URL, schema request/response, strategia di versioning, paginazione, formato degli errori, authoring specifiche OpenAPI. Carica la skill `rest-api-standards`.

Produce specifiche OpenAPI 3.1 YAML e rationale di design. Fa anche review di API esistenti per REST maturity level, consistenza, e rischio di breaking change.

Parte IV

Strumenti di contorno

Capitolo 21

Hooks

21.1 Cosa sono gli hook

Gli hook sono comandi shell (o endpoint HTTP, o prompt) che Claude Code esegue automaticamente in risposta a eventi specifici durante una sessione. Girano *fuori* dal contesto di Claude — sono script shell standard eseguiti dal processo Claude Code.

21.1.1 Proprietà chiave

- Gli hook `PreToolUse` sono **sincroni e bloccanti**: possono ritardare o bloccare la tool call
- L'**exit code** controlla il comportamento:
 - `exit 0` — consenti l'azione
 - `exit 2` — blocca l'azione (solo per hook `PreToolUse` bloccanti)
 - Qualsiasi altro exit code — fallimento hook, loggato ma l'azione prosegue
- Gli hook possono produrre JSON su stdout per comunicare decisioni strutturate a Claude Code
- Gli hook girano con lo stesso ambiente del processo Claude Code

21.2 Hook inclusi nel catalogo

21.2.1 `pre-tool-safety.sh`

Evento: `PreToolUse` su Bash. Scopo: warning prima di comandi shell distruttivi (`rm -rf`, `DROP TABLE`, ecc.). Comportamento: esce con 2 (blocca) se viene rilevato un pattern ad alto rischio, con una spiegazione.

21.2.2 post-session-log.sh

Evento: Stop. Scopo: aggiunge un timestamp di fine sessione a un audit log locale.

Comportamento: esce sempre con 0 (non bloccante). Scrive in `.claude/session-log.txt`.

21.3 Come attivare gli hook

Aggiungi le voci rilevanti al `.claude/settings.json` del progetto. Gli hook **non** si attivano posizionando gli script nella directory `hooks/`: quella directory contiene solo esempi e template. Devono essere configurati esplicitamente in `settings.json`.

```
# Esempio entry in .claude/settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "bash .claude/hooks/pre-tool-safety.sh"
          }
        ]
      }
    ]
  }
}
```

21.4 Linee guida per lo sviluppo di hook

1. **Tienili veloci:** hook che impiegano più di qualche secondo degradano significativamente la developer experience. Pre-tool hook in particolare devono completare in <500ms.
2. **Non bloccare per default:** blocca (`exit 2`) solo in presenza di una chiara preoccupazione di sicurezza. Hook bloccanti troppo aggressivi diventano rumore e vengono disattivati.
3. **Mai credenziali negli script hook:** usa variabili d'ambiente.
4. **Log su stderr, non stdout:** Claude Code può processare stdout per decisioni strutturate.
5. **Gestisci stdin vuoto o malformato:** gli script hook ricevono JSON su stdin; gestisci sempre il caso in cui stdin è vuoto.

Capitolo 22

MCP (Model Context Protocol)

22.1 Cos'è MCP

MCP (Model Context Protocol) è uno standard ufficiale Claude Code che permette di esporre tool e sorgenti dati esterni a Claude come strumenti invocabili. I server MCP girano come processi separati e comunicano con Claude Code via protocollo definito.

22.2 Quando usare MCP — e quando no

22.2.1 Usa MCP quando

- Hai bisogno che Claude acceda a una sorgente dati *fuori dalla directory di progetto* e non raggiungibile con Read/Grep/Glob (e.g. un database, un'API esterna, un wiki, una JIRA instance)
- L'integrazione viene riutilizzata su più sessioni — il costo di setup è giustificato
- Il tool deve restituire dati strutturati che Claude altrimenti dovrebbe estrarre da testo non strutturato

22.2.2 Non usare MCP quando

- I file sono nella directory di progetto — usa Read, Grep, Glob
- Hai bisogno di eseguire un comando shell — usa Bash
- Hai bisogno di una web lookup one-off — usa WebFetch
- Il setup del server MCP è più complesso del beneficio che porta

Buona pratica

La regola pratica: se puoi risolvere il problema con gli strumenti che Claude Code già ha (Read, Bash, WebFetch), non introdurre un server MCP. MCP aggiunge complessità operativa (un altro processo da avviare, mantenere, e proteggere).

22.3 Formato del file `.mcp.json`

`.mcp.json` va alla **root del progetto** (non dentro `.claude/`). È sicuro committarlo su Git purché non contenga segreti.

```
{
  "mcpServers": {
    "nome-server": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-name"],
      "env": {
        "API_KEY": "${API_KEY_ENV_VAR}"
      }
    }
  }
}
```

I riferimenti a variabili d'ambiente (`${VAR}`) vengono espansi dall'ambiente shell a runtime. Non inserire mai segreti direttamente nel file.

22.4 Server MCP comuni

Server	Package	Fornisce
Filesystem	@mcp/server-filesystem	Read/write su directory specificate
PostgreSQL	@mcp/server-postgres	Schema DB e accesso query
GitHub	@mcp/server-github	Issue, PR, ricerca codice
Brave Search	@mcp/server-brave-search	Web search

Capitolo 23

Script operativi

23.1 `setup-capabilities.sh`

Installa le capability dal marketplace nel progetto target (o globalmente con `--global`). Legge `catalog.json` per risolvere il path corretto di ogni capability e le relative dipendenze (skill). L'installazione è idempotente: chiamate successive sovrascrivono senza errori.

```
# Installazione globale
./claude-catalog/scripts/setup-capabilities.sh --global

# Installazione per progetto
./claude-catalog/scripts/setup-capabilities.sh /percorso/progetto
```

23.2 `new-capability.sh`

Scaffold una nuova capability (agente o skill) con tutti i file richiesti:

```
# Scaffold un nuovo agente
./claude-catalog/scripts/new-capability.sh --type agent nome-agente

# Scaffold una nuova skill
./claude-catalog/scripts/new-capability.sh --type skill nome-skill
```

Il template generato include: frontmatter YAML con i campi obbligatori, sezione `## When to invoke` con placeholder per 2-4 scenari, sezione `Do NOT use this agent for`, struttura `## Role`, e il pointer agli scenari worked.

23.3 `publish.sh`

Copia una capability approvata dal catalogo al marketplace e aggiorna `catalog.json`.

```
# Pubblica in beta (topic auto-risolto dal path nel catalogo)
```

```
./claude-marketplace/scripts/publish.sh nome-capability 0.1.0 beta

# Pubblica in stable con topic esplicito
./claude-marketplace/scripts/publish.sh software-architect 1.1.0 stable \
  --topic architecture
```

Lo script:

1. Risolve il topic da: flag `--topic`, entry esistente in `catalog.json`, path sorgente nel catalogo
2. Copia il file `.md` in `claude-marketplace/{tier}/{topic}/{name}.md`
3. Rimuove eventuali copie stantie della stessa capability sotto lo stesso tier
4. Aggiorna version, tier, file, e published in `catalog.json`

23.4 `validate_catalog.py` e `validate_marketplace.py`

Script di validazione eseguiti dalla CI — eseguibili anche localmente prima di aprire una PR:

```
# Gate 1 - valida il catalogo
python3 .github/scripts/validate_catalog.py

# Gate 2 - valida il marketplace (esegui solo dopo Gate 1 verde)
python3 .github/scripts/validate_marketplace.py
```

23.4.1 Cosa valida Gate 1 (catalogo)

- Frontmatter YAML valido (name, description, tools, model)
- System prompt presente con sezione `## Role`
- Skill senza strumenti vietati (Edit, Write, Bash, Agent)
- `CHANGELOG.md` ha una entry [Unreleased]
- **Check marketplace sync:** ogni agente/skill nel catalogo deve avere una entry in `catalog.json` — blocca la PR se manca

23.4.2 Cosa valida Gate 2 (marketplace)

- `catalog.json` valido (semver, tier, status, campi obbligatori)
- Ogni file referenziato esiste su disco
- Il campo name nel frontmatter corrisponde alla entry in catalogo
- Convenzione dei path: `{tier}/{topic}/{name}.md` o `skills/{topic}/{name}.md`
- Nessun file orfano in `stable/`, `beta/`, `skills/` (ricorsivo)

Capitolo 24

Template e policy

24.1 Template

Il catalogo include template riutilizzabili per output ricorrenti:

File	Scopo
templates/architecture-decision-record.md	Template ADR standard (Contesto, Decisione)
templates/analysis-report.md	Template report di analisi (Executive Summary)
templates/api-contract.md	Template contratto API con schema request/response

24.2 Policy

policies/python-conventions.md contiene le convenzioni di codifica Python adottate dal team (non ancora migrate a skill). Viene referenziata dagli agenti Python per garantire consistenza.

24.3 Evals

Ogni capability dovrebbe avere almeno due scenari di valutazione in evals/{capability}-eval.md. Gli eval descrivono: scenario di input, comportamento atteso, criteri di successo. Sono usati sia per il testing manuale prima del merge sia per future valutazioni automatizzate.

Appendice A

Riferimento rapido: comandi essenziali

```
# Aggiornare le capability globalmente
cd ~/dev/claude-registry && git pull origin main
./claude-catalog/scripts/setup-capabilities.sh --global

# Validare prima di aprire una PR
python3 .github/scripts/validate_catalog.py
python3 .github/scripts/validate_marketplace.py

# Scaffoldare una nuova capability
./claude-catalog/scripts/new-capability.sh --type agent nome-agente
./claude-catalog/scripts/new-capability.sh --type skill nome-skill

# Pubblicare una capability approvata
./claude-marketplace/scripts/publish.sh nome 0.1.0 beta

# Versione pubblicata di una capability specifica
cat claude-marketplace/catalog.json | \
python3 -c "import json,sys; \
[print(c['name'], c['version']) \
for c in json.load(sys.stdin)['capabilities'] \
if c['name']=='nome-capability']"
```


Appendice B

Elenco completo delle capability

B.1 Agenti per area

Area	Agente	Descrizione sintetica
<i>Orchestrazione</i>	orchestrator	Meta-orchestratore multi-dominio
	refactoring-supervisor	Workflow replatforming end-to-end (5 fasi)
<i>Deliberazione</i>	deliberative-decision-engine	Motore di decisione deliberativa a 7 step
	debate-proposer	Persona: Primary Architect
	debate-critic	Persona: Skeptical Critic
	debate-replatforming-specialist	Persona: Migration Specialist
	debate-risk-reviewer	Persona: Risk/Compliance Reviewer
	debate-operations-reviewer	Persona: Operations Reviewer
	debate-judge	Giudice neutrale e arbitro
<i>Phase 0 — Indicizzazione</i>	indexing-supervisor	Supervisore pipeline di indicizzazione
	codebase-mapper	Inventario strutturale
	dependency-analyzer	Dipendenze e grafo importazioni
	streamlit-analyzer	Analisi specifica Streamlit
	module-documenter	Documentazione API per package
	data-flow-analyst	Flussi dati e boundary esterni
	business-logic-analyst	Regole di business e domain concepts
	synthesizer	Sintesi finale KB
<i>Phase 1 — Analisi funzionale</i>	functional-analysis-supervisor	Supervisore Phase 1
	actor-feature-mapper	Attori e feature map

Area	Agente	Descrizione sintetica
	ui-surface-analyst	Schermate e navigazione
	io-catalog-analyst	Input/output e trasformazioni
	user-flow-analyst	Use case e user flow
	implicit-logic-analyst	Logica implicita e state machine
	functional-analysis-challenger	Revisione avversariale
<i>Phase 2 — Analisi tecnica</i>		
	technical-analysis-supervisor	Supervisore Phase 2
	code-quality-analyst	Qualità codice e hotspot
	state-runtime-analyst	Session state e side effect
	dependency-security-analyst	Dipendenze, CVE, licenze
	data-access-analyst	Pattern accesso dati
	integration-analyst	Integrazioni esterne
	performance-analyst	Performance statica
	resilience-analyst	Error handling e silent failure
	security-analyst	OWASP Top 10 e STRIDE
	risk-synthesizer	Registro di rischio unificato
	technical-analysis-challenger	Revisione avversariale
<i>Phase 3 — Baseline testing</i>		
	baseline-testing-supervisor	Supervisore Phase 3
	fixture-builder	confest.py e fixture
	usecase-test-writer	Test per use case
	integration-test-writer	Test di integrazione
	benchmark-writer	Oracle performance
	service-collection-builder	Postman collection
	baseline-runner	Esecuzione suite + oracle
	baseline-challenger	Revisione avversariale
<i>Phase 4 — Refactoring TO-BE</i>		
	decomposition-architect	Bounded context + ADR-001/002
	api-contract-designer	OpenAPI 3.1 + ADR-003
	backend-scaffolder	Spring Boot 3 scaffold
	data-mapper	JPA entities + Liquibase
	logic-translator	Traduzione UC Python→Java
	frontend-scaffolder	Angular 17+ workspace
	hardening-architect	Observability + security + ADR-004/005
	migration-roadmap-builder	Strangler-fig roadmap
	phase4-challenger	Traceability matrix + 8 check
<i>Developer</i>		
	developer-java	Java/Spring Boot 3
	developer-python	Python/FastAPI
	developer-frontend	Angular/React/Vue/Qwik/Vanilla
	developer-go	Go
	developer-kotlin	Kotlin/Spring Boot o Ktor

Area	Agente	Descrizione sintetica
	developer-rust	Rust/axum/tokio
	developer-csharp	C#/.NET 8+
	developer-ruby	Ruby/Rails 7+
	developer-php	PHP 8.2+/Laravel o Symfony
<i>Qualità</i>	code-reviewer	Code review strutturata
	debugger	Diagnosi bug e root cause
	test-writer	Scrittura test
	registry-auditor	Audit rubric Anthropic
<i>Documentazione</i>	documentation-writer	README, runbook, guide API
	wiki-writer	GitHub wiki (Diataxis)
	document-creator	PDF/DOCX Accenture-branded
	presentation-creator	PPTX Accenture-branded
<i>Analisi e design</i>	functional-analyst	Requisiti, use case, business process
	technical-analyst	Debito tecnico, sicurezza, CI/CD
	software-architect	Architettura, ADR, trade-off
	api-designer	REST API design, OpenAPI 3.1

Appendice C

Glossario

Agente (Agent) Sottoagente Claude Code che svolge un ruolo attivo (analizza, scrive, progetta). Ha un system prompt con logica comportamentale, strumenti appropriati, e una description che guida la delega automatica.

Skill Fornitore di conoscenza dichiarativa (standard, convenzioni, regole). Nodo foglia — non delega ad altri agenti, non modifica file. Modello haiku, strumento solo Read.

Capability Il pacchetto completo: agente o skill + documentazione + esempi + eval.

Catalogo L'area di sviluppo e versionamento delle capability (claude-catalog/).

Marketplace L'area di distribuzione delle capability approvate (claude-marketplace/).

Oracle Insieme degli snapshot catturati dell'applicazione AS-IS (output per use case, performance) usati come riferimento di equivalenza in Phase 5.

Strangler-fig Pattern di migrazione progressiva: il sistema legacy viene gradualmente "strangolato" da componenti TO-BE che ne sostituiscono le funzionalità una alla volta.

HITL (Human-in-the-loop) Punto obbligatorio di conferma umana in un flusso automatizzato.

AS-IS Lo stato attuale dell'applicazione, senza modifiche o riferimenti al target.

TO-BE L'architettura e l'implementazione target della migrazione/refactoring.

DDD (Domain-Driven Design) Approccio di design basato su bounded context, aggregati, e linguaggio ubiquo.

ADR (Architecture Decision Record) Documento che registra una decisione architetture con il suo contesto, la decisione presa, e le conseguenze.

Semver Versionamento semantico: MAJOR.MINOR.PATCH con regole precise per i bump.

Fan-out Dispatching parallelo di più sub-agent per task indipendenti (e.g., un test writer per use case in parallelo).